Universidad Pontificia Comillas · ICAI
Grado en Ingeniería Matemática e Inteligencia Artificial

GPS de Madrid

Cálculo de rutas sobre el grafo de calles de la ciudad

Memoria técnica · Práctica 3 de Matemática Discreta

Grupo GP11B
Integrantes Jaime Tuesta de Simón
Ignacio Mascarell Agúndez
Asignatura Matemática Discreta (2.º curso)
Fecha Noviembre de 2025

Índice

1 Introducción **2**

2 Arquitectura del programa **2**

3 Datos: callejero y grafo de calles **3**

3.1 Lectura del callejero . . . 3

3.2 Búsqueda de direcciones . . . 3

3.3 Búsqueda aproximada . . . 4

3.4 Grafo de calles . . . 4

4 Algoritmos sobre grafos pesados **5**

5 Funciones de peso **6**

5.1 Ruta más corta . . . 6

5.2 Ruta más rápida . . . 6

5.3 Ruta más rápida con semáforos . . . 7

5.4 Comparación de los tres modos . . . 7

6 Instrucciones de navegación **8**

6.1 Agrupación por vía . . . 8

6.2 Tipo de giro . . . 8

6.3 Redacción de los mensajes . . . 9

7 Representación gráfica de la ruta **10**

8 Limitaciones **11**

9 Uso de herramientas de inteligencia artificial **11**

Referencias **11**

1. Introducción

Este documento describe y justifica el procedimiento seguido para resolver la Práctica 3 de la asignatura de Matemática Discreta: el desarrollo de un sistema de navegación (GPS) para la ciudad de Madrid. El programa recibe una dirección de origen y otra de destino, calcula la ruta óptima entre ambas según el criterio elegido por el usuario y la presenta mediante instrucciones de navegación y un mapa con el recorrido resaltado.

La solución modela la red viaria como un grafo dirigido y pesado, en el que los nodos representan intersecciones y extremos de calle, y las aristas, los tramos que los unen. Sobre este grafo se aplica el algoritmo de Dijkstra con tres funciones de peso distintas: la longitud del tramo, su tiempo de recorrido y ese mismo tiempo más una penalización por semáforos.

A lo largo de la memoria se utiliza como ejemplo una ruta que cruza la ciudad de norte a sur, desde el Paseo de la Castellana, 144 (junto al estadio Santiago Bernabéu) hasta el Paseo de la Chopera, 14 (Matadero Madrid). Todas las figuras de mapas y de consola proceden de ejecuciones reales del programa.

2. Arquitectura del programa

El proyecto se organiza en cuatro módulos de Python con responsabilidades diferenciadas, que se resumen en la Tabla 1. La Figura 1 muestra cómo fluyen los datos entre ellos para resolver una consulta.

Tabla 1. Módulos del programa y responsabilidades.

Módulo	Responsabilidad
`callejero.py`	Lectura y tratamiento del callejero de Madrid, búsqueda de direcciones (exacta y aproximada) y obtención del grafo de calles.
`gps.py`	Programa principal: interacción con el usuario, cálculo de la ruta, instrucciones de navegación y representación gráfica.
`grafo_pesado.py`	Biblioteca de algoritmos sobre grafos pesados: Dijkstra, reconstrucción del camino mínimo, Prim y Kruskal.
`pesos.py`	Funciones de peso de las aristas: longitud, tiempo de recorrido y tiempo con semáforos.

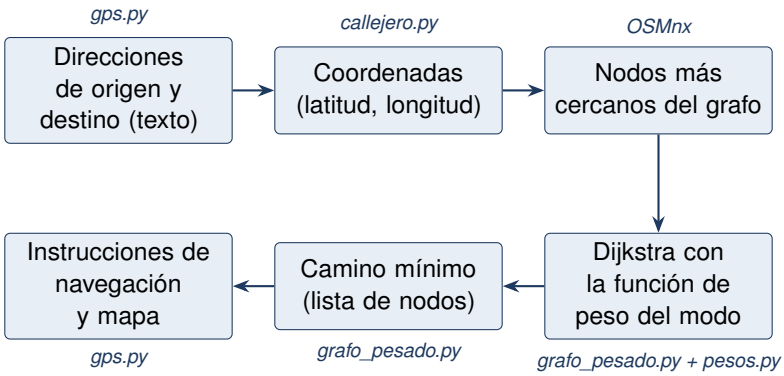


Figura 1. Flujo de una consulta: de las direcciones escritas por el usuario a la ruta. En cursiva, el módulo o la librería responsable de cada paso.

El programa principal funciona en bucle: tras mostrar una ruta, vuelve a pedir direcciones, y finaliza cuando el usuario introduce una dirección vacía.

3. Datos: callejero y grafo de calles

3.1 Lectura del callejero

El programa lee el fichero de direcciones del callejero oficial del Ayuntamiento de Madrid [4]. Durante la lectura se seleccionan únicamente las columnas necesarias (tipo de vía, partícula, nombre, número, latitud y longitud), lo que reduce de forma significativa el coste temporal y espacial. El resultado es un `DataFrame` de `pandas`, sobre el que se realizan dos ajustes: los valores ausentes se sustituyen por un marcador (`'X'`), y las coordenadas, que el fichero expresa en grados, minutos y segundos, se convierten a grados decimales. Para ello, una expresión regular separa los cuatro componentes de cada coordenada y se aplica

$$\text{valor} = \pm\left(g + \frac{m}{60} + \frac{s}{3600}\right),$$

con signo negativo para las orientaciones oeste (W) y sur (S). Si el fichero no se encuentra, el programa informa del error.

3.2 Búsqueda de direcciones

La búsqueda se realiza filtrando el `DataFrame` con las herramientas de `pandas`. Para identificar los elementos de la dirección introducida, que se pasa a mayúsculas, se emplean dos expresiones regulares:

- **Formato completo**, con tipo de vía y partícula opcional: *Calle de Alberto Aguilera, 23*. La primera palabra solo se acepta como tipo de vía si pertenece a la lista de tipos válidos (calle, avenida, paseo, plaza, glorieta, etc.); así, un nombre sin tipo de vía no se interpreta por error en este formato.
- **Formato simplificado**, solo con el nombre y el número: *Alberto Aguilera, 23*.

Si una dirección corresponde a varias ubicaciones (por ejemplo, el mismo nombre con distintos tipos de vía en el formato simplificado), se pide al usuario que elija una. Si la dirección no tiene

un formato válido, se informa del formato esperado.

3.3 Búsqueda aproximada

Cuando la dirección no aparece en el callejero, en lugar de devolver un error se buscan las direcciones más parecidas (*fuzzy search*). La similitud se mide con la distancia de Levenshtein $d(a, b)$: el número mínimo de inserciones, borrados y sustituciones de caracteres necesarios para transformar una cadena en otra. Se calcula por programación dinámica con la recurrencia

$$D_{i,j} = \min\big(D_{i-1,j} + 1,\ D_{i,j-1} + 1,\ D_{i-1,j-1} + [a_i \neq b_j]\big),$$

guardando solo la fila anterior y la actual de la matriz, y se normaliza por la longitud de la cadena más larga para obtener un valor entre 0 y 1:

$$\hat{d}(a, b) = \frac{d(a, b)}{\max(|a|, |b|)}.$$

La entrada se compara con todas las direcciones del callejero, escritas en el mismo formato que el usuario (completo o simplificado). Se descartan las que tienen $\hat{d} \geq 0{,}4$ y se proponen las cinco más cercanas; el usuario puede elegir una o descartarlas todas (opción 0) y volver a escribir la dirección. La Figura 2 muestra un ejemplo con una errata en el nombre de la calle: la primera sugerencia es la dirección buscada, y el resto son direcciones de la misma calle con números parecidos, ya que la distancia también cuenta las diferencias en el número.

```
Introduzca la dirección de origen:
 - Dirección: Paseo de la Castellsna, 114
La dirección no se encuentra en la base de datos. ¿Quiere decir...?
1. PASEO DE LA CASTELLANA, 114
2. PASEO DE LA CASTELLANA, 11
3. PASEO DE LA CASTELLANA, 14
4. PASEO DE LA CASTELLANA, 104
5. PASEO DE LA CASTELLANA, 110
Elija la opción que desee (o 0 si ninguna):
```

Figura 2. Búsqueda aproximada: la dirección «Paseo de la Castellsna, 114» no existe en el callejero y el programa propone las cinco direcciones más parecidas.

3.4 Grafo de calles

El grafo de la red viaria de Madrid se obtiene de OpenStreetMap [5] mediante la librería OSMnx [2], con la red de vías transitables en coche. La primera ejecución descarga el grafo y lo guarda en un fichero GraphML; las siguientes lo cargan desde dicho fichero.

OSMnx devuelve un multigrafo dirigido (`MultiDiGraph`) ya simplificado: los nodos intermedios de una calle se eliminan y solo se conservan intersecciones y extremos, de modo que un mismo tramo puede reunir varios nombres o velocidades en forma de lista. El programa lo transforma en un grafo dirigido simple (`DiGraph`): cuando hay aristas paralelas entre dos nodos, se conserva la de menor longitud. Por último, se eliminan los bucles (aristas de un nodo a sí mismo), que no aportan nada a un camino mínimo. Cada nodo guarda sus coordenadas (`x` = longitud, `y` = latitud) y cada arista, atributos como la longitud en metros (`length`), el nombre de la vía (`name`), su referencia (`ref`), el tipo de vía (`highway`) y la velocidad máxima (`maxspeed`). La Figura 3 muestra el grafo resultante.



Figura 3. Grafo de calles de Madrid utilizado por el GPS, dibujado a partir de las coordenadas de sus nodos.

4. Algoritmos sobre grafos pesados

La biblioteca `grafo_pesado.py` implementa los algoritmos del tema de grafos de la asignatura [1]. Todas las funciones reciben como parámetro una *función de peso* $w(G, u, v)$ que devuelve el coste de la arista (u, v), lo que permite reutilizarlas con cualquier criterio sin modificar el algoritmo.

Dijkstra. Calcula el árbol de caminos mínimos desde un vértice de origen. Mantiene para cada vértice su distancia provisional, su padre en el árbol y si ya ha sido visitado. En cada iteración extrae de una cola de prioridad (montículo binario del módulo `heapq`) el vértice no visitado con menor distancia, lo marca como visitado y relaja sus aristas salientes:

$$\text{si } \operatorname{dist}(u) + w(u,v) < \operatorname{dist}(v): \quad \operatorname{dist}(v) \leftarrow \operatorname{dist}(u) + w(u,v), \;\; \operatorname{padre}(v) \leftarrow u.$$

Cuando la distancia de un vértice mejora, se inserta de nuevo en el montículo con la nueva prioridad, y las entradas antiguas se ignoran al extraerlas porque el vértice ya está visitado. El algoritmo es correcto porque todos los pesos son no negativos. Con el montículo, su coste es $O((V + E) \log V)$, frente al $O(V^2)$ de buscar el mínimo recorriendo todos los vértices, lo que resulta imprescindible en un grafo del tamaño del de Madrid.

Camino mínimo. `camino_minimo`, la función que utiliza `gps.py`, ejecuta Dijkstra desde el origen y reconstruye el camino recorriendo los padres desde el destino hasta el origen; después invierte la lista. Si el destino no tiene padre en el árbol, no es alcanzable desde el origen (por ejemplo, por calles de sentido único), la función devuelve `None` y el programa informa de que no existe una ruta en coche.

Prim y Kruskal. El módulo implementa también estos dos algoritmos, que no intervienen en el cálculo de rutas. Ambos obtienen un árbol abarcador mínimo: un conjunto de aristas que conecta todos los vértices sin formar ciclos y con coste total mínimo. Difieren en su enfoque: el algoritmo de Prim hace crecer el árbol desde un vértice inicial, añadiendo en cada paso la arista más barata que lo conecta con un vértice nuevo, mientras que el de Kruskal recorre las aristas en orden creciente de peso y añade cada una siempre que una dos componentes distintas, es decir, siempre que no genere un ciclo. Es importante no confundir ambos problemas: un árbol abarcador mínimo minimiza el coste total de conectar *todos* los vértices, mientras que un camino mínimo minimiza el coste entre *dos* vértices concretos, que es lo que necesita un GPS.

5. Funciones de peso

El archivo `pesos.py` define las funciones de coste que asignan un valor numérico a cada arista (u, v) del grafo. El usuario elige el criterio de optimización entre las tres opciones de la Tabla 2; en todos los casos, Dijkstra minimiza la suma de los pesos de las aristas del camino.

Tabla 2. Modos de cálculo de ruta y funciones de peso asociadas.

Modo	**Función**	**Peso de la arista** (u, v)
Más corta (MC)	`length_weight`	$\ell(u, v)$, en metros.
Más rápida (MR)	`time_weight`	$t(u, v) = \ell(u, v)/v_{\max}(u, v)$, en segundos.
Más rápida con semáforos (MRO)	`stoplights_weight`	$t(u, v)$ más 24 s si v es una intersección.

5.1 Ruta más corta

El peso es la longitud física del tramo, $w_{\mathrm{MC}}(u, v) = \ell(u, v)$, tomada del atributo `length` de la arista. El camino óptimo es, por tanto, el de menor distancia recorrida.

5.2 Ruta más rápida

El peso es el tiempo necesario para recorrer el tramo a la velocidad máxima permitida:

$$w_{\mathrm{MR}}(u, v) = t(u, v) = \frac{\ell(u, v)}{v_{\max}(u, v) \cdot \frac{1000}{3600}} \quad [\mathrm{s}],$$

con ℓ en metros y $v_{\max}$ en km/h. La velocidad máxima de cada arista se determina así:

1. Si la arista tiene el atributo `maxspeed`, se extrae con una expresión regular el primer número de su valor (los valores de OpenStreetMap pueden incluir texto, como «50 mph» o «ES:urban»). Si el tramo agrupa varias velocidades, se toma la mayor, suponiendo que la vía se rige por la menos restrictiva.
2. Si no tiene `maxspeed`, o su valor no contiene ningún número, se asigna una velocidad según el tipo de vía (`highway`), de acuerdo con la Tabla 3. Si el tramo agrupa varios tipos de vía, se toma la mayor de sus velocidades.
3. Si tampoco el tipo de vía figura en la tabla, se usa un valor por defecto de 30 km/h.

Tabla 3. Velocidad asignada según el tipo de vía cuando la arista no indica su velocidad máxima.

Tipo de vía	km/h	Tipo de vía	km/h
`living_street`	20	`secondary`, `tertiary`, `primary`	50
`residential`	30	`trunk`, `tertiary_link`, `busway`	50
`unclassified`	40	`motorway_link`	70
`primary_link`, `secondary_link`, `trunk_link`	40	`motorway`	100

5.3 Ruta más rápida con semáforos

Este modo añade al tiempo de recorrido una penalización por la espera prevista al llegar a una intersección, según el modelo del enunciado:

$$w_{\mathrm{MRO}}(u,v) = t(u,v) + p(v)\cdot T, \qquad p(v) = \begin{cases} 0{,}8 & \text{si } \deg^-(v) + \deg^+(v) > 2, \\ 0 & \text{en otro caso,} \end{cases} \qquad T = 30 \text{ s}.$$

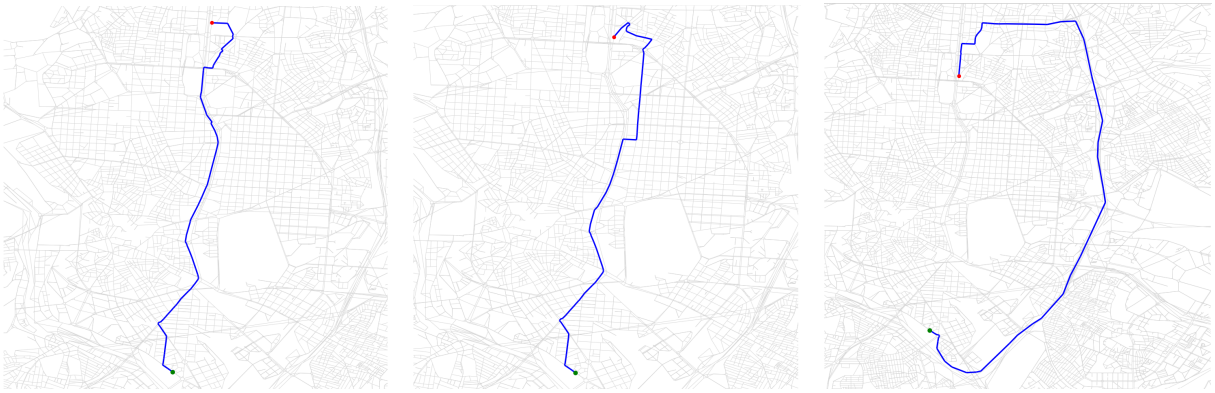
La penalización se interpreta como un tiempo esperado: en cada intersección hay un semáforo que se encuentra en rojo con probabilidad 0,8, y en ese caso la espera es de 30 s. Cada intersección añade así $0{,}8 \cdot 30 = 24$ s al coste del camino. Se considera intersección todo nodo con más de dos aristas incidentes, contando tanto las entrantes ($\deg^-$) como las salientes ($\deg^+$). Un nodo con dos o menos, como el extremo de una calle sin salida de doble sentido (una arista de entrada y otra de salida), no se penaliza. Como el grafo está simplificado, prácticamente el resto de nodos son cruces entre calles.

La penalización se aplica al nodo de llegada v de cada arista, por lo que un camino de k aristas que atraviesa n intersecciones paga $24n$ segundos además de su tiempo de recorrido. Este modo prefiere, por tanto, vías con pocos cruces aunque sean más largas.

5.4 Comparación de los tres modos

La Figura 4 muestra la ruta de ejemplo calculada con cada modo. Las tres respuestas son distintas y se explican por la función de peso:

- **MC** sigue el eje Castellana–Recoletos–Prado y atraviesa el centro: es la opción más directa. Según las instrucciones de la Figura 6, recorre unos 7,9 km.
- **MR** baja por la calle de Serrano, paralela a la Castellana: el criterio ya no es la distancia sino el tiempo, que depende de la velocidad máxima de cada tramo.
- **MRO** evita el centro y rodea la ciudad por el este, recorriendo unos 8 km por la Avenida de la Paz (tramo este de la M-30). Aunque la ruta es bastante más larga, la vía tiene muchas menos intersecciones que la trama urbana y una velocidad mayor, de modo que la penalización de 24 s por cruce hace que este rodeo resulte el camino de menor coste.



(a) Más corta (MC). **(b)** Más rápida (MR). **(c)** Con semáforos (MRO).

Figura 4. Ruta entre el Paseo de la Castellana, 144 (punto rojo) y el Paseo de la Chopera, 14 (punto verde) con cada modo de cálculo. Cada mapa se ajusta a su propia ruta, por lo que la escala no es la misma en los tres.

6. Instrucciones de navegación

A partir de la lista de nodos del camino, el programa genera instrucciones legibles. El proceso tiene tres partes: agrupar los tramos que pertenecen a una misma vía, decidir el tipo de giro en cada cambio de vía y redactar el mensaje correspondiente.

6.1 Agrupación por vía

Se recorren las aristas del camino en orden. Para cada arista se obtiene el nombre de la vía: el atributo `name` si existe (si el tramo agrupa varios nombres, se unen en la forma «A, B y C»), la referencia `ref` si no tiene nombre, o la marca `NO-NAME` si no tiene ninguno de los dos. La longitud de cada arista se acumula mientras el nombre se mantiene. Cuando la siguiente arista tiene un nombre distinto, o se llega a la última arista, se emite una instrucción con la distancia acumulada y el contador se reinicia. Así, una calle formada por muchos tramos genera una sola instrucción.

6.2 Tipo de giro

En cada cambio de vía se clasifica el giro a partir de los tres nodos implicados: u y v, extremos de la última arista de la vía actual, y w, final de la primera arista de la nueva vía. Con sus coordenadas se forman los vectores

$$\mathbf{a} = v - u = (a_x, a_y), \qquad \mathbf{b} = w - v = (b_x, b_y),$$

y se calcula el ángulo entre ellos y la componente z de su producto vectorial:

$$\theta = \arccos \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}, \qquad \mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x.$$

Si $\theta < 30^\circ$, la dirección apenas cambia y se considera que el vehículo sigue recto. En otro caso, el signo del producto vectorial indica el sentido del giro: positivo si $\mathbf{b}$ queda a la izquierda de $\mathbf{a}$ (giro antihorario) y negativo si queda a la derecha (Figura 5). Como el redondeo puede dar cocientes ligeramente mayores que 1 en valor absoluto, el coseno se limita al intervalo $[-1, 1]$ antes de calcular el arcocoseno.

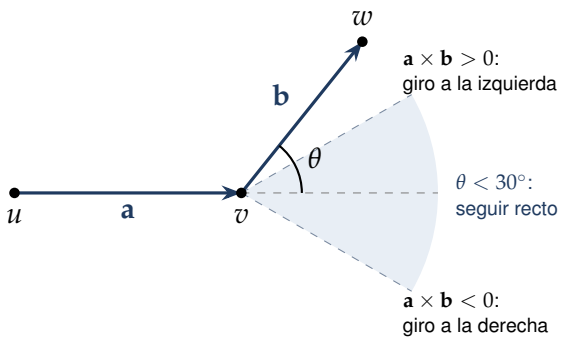


Figura 5. Clasificación del giro en el nodo v. Si $\mathbf{b}$ cae dentro de la zona sombreada ($\theta < 30^\circ$) se sigue recto; fuera de ella, el signo de $\mathbf{a} \times \mathbf{b}$ distingue la izquierda de la derecha. En el ejemplo, $\theta \approx 51^\circ$ y el producto vectorial es positivo: giro a la izquierda.

6.3 Redacción de los mensajes

Con el nombre de la vía actual, el de la siguiente y el tipo de giro, el mensaje se elige según los casos de la Tabla 4. Las calles sin nombre se tratan aparte porque no se puede decir «por» qué vía se avanza; en ese caso la distancia y la maniobra se combinan en un único mensaje.

Tabla 4. Mensajes generados en cada caso (X: distancia acumulada; N_1: vía actual; N_2: vía siguiente).

Vía actual	Situación	Mensaje
Con nombre	Último tramo	«Avance X m por N_1 hasta llegar a su destino.»
	Giro a izquierda o derecha	«Avance X m por N_1.» y «Gire a la izquierda/derecha hacia N_2.» (sin «hacia N_2» si la vía siguiente no tiene nombre)
	Cambio de vía en recto	«Avance X m por N_1.» y «Siga recto hacia N_2.» (este último solo si la vía siguiente tiene nombre)
Sin nombre	Último tramo	«Avance X m hasta llegar a su destino.»
	Giro a izquierda o derecha	«Avance X m y gire a la izquierda/derecha hacia N_2.»
	Cambio de vía en recto	«Siga recto X m hasta N_2.»

La Figura 6 muestra las instrucciones de la ruta de ejemplo en modo MC. Se aprecia la agrupación: los 2,4 km de la Castellana aparecen en dos instrucciones, separadas solo por el paso por la Plaza del Doctor Marañón, que en OpenStreetMap es una vía distinta. Lo mismo ocurre en las plazas de Colón y del Emperador Carlos V, que se atraviesan en recto.

```
DIRECCIONES:
Avance 238.34 m por Calle de Rafael Salgado.
Gire a la derecha hacia Calle del Padre Damián.
Avance 228.28 m por Calle del Padre Damián.
Gire a la derecha hacia Glorieta de los Sagrados Corazones.
Avance 95.38 m por Glorieta de los Sagrados Corazones.
Gire a la derecha hacia Paseo de La Habana.
Avance 265.81 m por Paseo de La Habana.
Gire a la izquierda hacia Calle de Santiago Bernabéu.
Avance 42.02 m por Calle de Santiago Bernabéu.
Gire a la derecha hacia Calle de Balvina Valverde.
Avance 385.52 m por Calle de Balvina Valverde.
Gire a la derecha hacia Calle de Joaquín Costa.
Avance 136.49 m por Calle de Joaquín Costa.
Gire a la izquierda hacia Paseo de la Castellana.
Avance 922.51 m por Paseo de la Castellana.
Siga recto hacia Plaza del Doctor Marañón.
Avance 38.46 m por Plaza del Doctor Marañón.
Siga recto hacia Paseo de la Castellana.
Avance 1443.31 m por Paseo de la Castellana.
Siga recto hacia Plaza de Colón.
Avance 49.84 m por Plaza de Colón.
Siga recto hacia Paseo de Recoletos.
Avance 724.98 m por Paseo de Recoletos.
Siga recto hacia Paseo del Prado.
Avance 1157.60 m por Paseo del Prado.
Siga recto hacia Plaza del Emperador Carlos V.
Avance 119.87 m por Plaza del Emperador Carlos V.
Siga recto hacia Paseo de Santa María de la Cabeza y Plaza del Emperador Carlos V.
Avance 105.12 m por Paseo de Santa María de la Cabeza y Plaza del Emperador Carlos V.
Siga recto hacia Paseo de Santa María de la Cabeza.
Avance 804.28 m por Paseo de Santa María de la Cabeza.
Gire a la derecha hacia Glorieta de Santa María de la Cabeza.
Avance 94.60 m por Glorieta de Santa María de la Cabeza.
Gire a la derecha hacia Calle de Embajadores.
Avance 240.17 m por Calle de Embajadores.
Gire a la derecha hacia Calle de Jaime I El Conquistador.
Avance 552.69 m por Calle de Jaime I El Conquistador.
Siga recto hacia Plaza del General Maroto.
Avance 54.51 m por Plaza del General Maroto.
Siga recto hacia Paseo de la Chopera.
Avance 170.78 m por Paseo de la Chopera hasta llegar a su destino.
```

Figura 6. Instrucciones de navegación generadas para la ruta de ejemplo en modo MC.

7. Representación gráfica de la ruta

Para facilitar la interpretación de la ruta, se dibuja el grafo de calles con las funciones de NetworkX y Matplotlib, usando como posición de cada nodo sus coordenadas, y se resalta el camino obtenido en azul, con el origen en rojo y el destino en verde. Para ampliar la zona de interés, se calcula el recuadro mínimo que contiene todos los nodos del trayecto, se convierte en un cuadrado centrado en él con un 10 % de margen y se ajustan los límites de los ejes a ese cuadrado, manteniendo la relación de aspecto 1:1. Los mapas de la Figura 4 son el resultado de esta función.

8. Limitaciones

- El modelo de semáforos es el del enunciado: penaliza por igual todas las intersecciones, tengan o no semáforo, y no tiene en cuenta el tráfico real.
- El ángulo de giro se calcula directamente sobre longitud y latitud. A la latitud de Madrid, un grado de longitud mide unas tres cuartas partes de un grado de latitud, por lo que los ángulos se deforman ligeramente; para clasificar giros con un umbral de 30° el efecto es pequeño.
- El giro se evalúa solo con los tramos inmediatamente anterior y posterior al cambio de vía, y solo se anuncian las maniobras en los cambios de nombre: una curva pronunciada dentro de una misma calle no genera instrucción.
- La búsqueda aproximada compara la entrada con todas las direcciones del callejero, por lo que es más lenta que la búsqueda exacta; solo se ejecuta cuando esta falla.

9. Uso de herramientas de inteligencia artificial

Durante la elaboración de la práctica se utilizaron herramientas de inteligencia artificial con los siguientes fines: pulir el código, corregir errores y resolver dudas puntuales. En particular, sirvieron de apoyo para utilizar las librerías de grafos, corregir pequeños errores del código y aclarar dudas sobre los algoritmos de Prim, Kruskal y Dijkstra. Su papel fue exclusivamente de apoyo: todo el trabajo entregado fue realizado por los integrantes del grupo.

Referencias

[1] Matemática Discreta, *Transparencias del Tema 6*. Universidad Pontificia Comillas (ICAI).

[2] G. Boeing. *OSMnx documentation*. https://osmnx.readthedocs.io

[3] NetworkX developers. *Tutorial — NetworkX documentation*. https://networkx.org/documentation/stable/tutorial.html

[4] Ayuntamiento de Madrid. *Callejero oficial del Ayuntamiento de Madrid: relación de direcciones vigentes, con coordenadas*. Portal de Datos Abiertos, licencia CC BY 4.0. https://datos.madrid.es

[5] OpenStreetMap contributors. *OpenStreetMap*, datos bajo licencia ODbL. https://www.openstreetmap.org